

**Автономная некоммерческая организация высшего образования
«Университет Иннополис»**

**ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
(БАКАЛАВРСКАЯ РАБОТА)
по направлению подготовки
09.03.01 - «Информатика и вычислительная техника»**

**GRADUATION THESIS
(BACHELOR'S GRADUATION THESIS)**

**Field of Study
09.03.01 – «Computer Science»**

**Направленность (профиль) образовательной программы
«Информатика и вычислительная техника»
Area of Specialization / Academic Program Title:
«Computer Science»**

**Тема
/
Topic**

**Разработка SDK для профилирования производительности и
обнаружения аномалий в Android-приложениях **
**Developing a Performance Profiling and Anomaly Detection SDK
for Android Applications**

Работу выполнил /
Thesis is executed by

**Миннемуллин Наиль
Фидаилевич /**
Nail Minnemullin

подпись / signature

**Руководитель
выпускной
квалификационной
работы /**
**Supervisor of
Graduation Thesis**

Маццара Мануэль /
Manuel Mazzara

подпись / signature

Иннополис, Innopolis, 2025

Contents

1	Introduction	7
1.1	Background and motivation	7
1.2	Research Gap	8
1.3	Research Questions and Aim	8
1.4	Novelty and Contribution	9
1.5	Structure	9
2	Literature Review	11
2.1	Performance monitoring: Concepts and Context	12
2.2	Techniques for Performance Data Collection	13
2.2.1	Pre-release Profiling Techniques	13
2.2.2	Post-release Monitoring Techniques	14
2.3	Performance Metrics: What to Measure and Why	16
2.4	Anomaly detection	18
2.5	Anomaly Detection in Performance Data	18
2.5.1	Mobile-Specific Gaps and Considerations	19
2.6	Summary	20
3	Methodology	22

CONTENTS	3
3.1 Introduction	22
3.2 System requirements and constraints	22
3.3 High-Level System Architecture	23
3.4 Android SDK Design	24
3.4.1 Metrics Selection and Collection	24
3.5 Backend Server Design	28
3.5.1 Data Storage	28
3.5.2 Anomaly Detection Algorithms	30
3.6 Testing and Validation	35
3.7 Summary	35
4 Implementation	36
5 Evaluation and Discussion	37
6 Conclusion	38
Bibliography cited	39

List of Tables

List of Figures

3.1	High-Level System Architecture	24
3.2	Sequence Diagram of SDK Data Collection and Transmission .	28

Abstract

Chapter 1

Introduction

1.1 Background and motivation

Mobile applications are now an essential part of everyday life, and users expect them to be fast, responsive, and stable. Poor performance (e.g. slow startup, high CPU usage, or freezing UI) can quickly degrade the user experience and result in negative reviews [1], [2]. Due to this reason, performance monitoring has become a crucial part of modern software development. To maintain high application quality, developers rely on profiling and monitoring tools that help them understand how applications behave under different conditions.

Performance analysis methods for mobile applications can be categorized into two main categories: pre-release profiling and post-release monitoring. Pre-release profiling provides detailed insights under controlled laboratory conditions but cannot accurately represent the wide variety of real devices and user behaviours. This limitation is especially critical because of Android fragmentation - the large variety of devices, operating system versions, and hardware configurations. The same application may perform differently across devices, which makes

lab-based testing results less reliable [3], [4]. In contrast, post-release monitoring reflects actual user conditions but often lacks automated alerting mechanisms, requiring developers to constantly monitor these metrics in order to notice performance issues in time. As a result, they need a way to continuously monitor how an application behaves on real devices under real usage conditions, with the ability to detect anomalies automatically.

1.2 Research Gap

Current research and existing tools focus on either pre-release (e.g., multi-layer profiling, energy or storage analysis) [5]–[7] or post-release monitoring and tracing in the wild [8]–[11]. However, none of them integrate anomaly detection methods. At the same time, studies on anomaly detection are developed independently and are not connected with the continuous monitoring of Android applications [12]–[14]. As a result, there is no existing solution that combines continuous post-release performance monitoring with cloud-based anomaly detection. This absence defines the main research gap that the current work aims to address.

1.3 Research Questions and Aim

The aim of this thesis is to develop a Software Development Kit (SDK) to continuously collect performance metrics from running Android application and detect anomalies in them. The SDK will collect key metrics directly from users' devices, send data to a backend, and notify developers when anomalous behaviour is detected.

The study addresses the following research questions:

1. How can continuous, low-overhead performance monitoring be implemented for Android applications on real user device?
2. Which metrics (such as CPU, RAM, FPS, and startup time) are most suitable for identifying performance anomalies that can downgrade user experience?
3. How can anomaly detection methods be applied to detect anomalies in performance metrics time series?
4. What is the practical effectiveness of the proposed SDK when applied to real or test applications?

1.4 Novelty and Contribution

The novelty of this work lies in combining post-release monitoring with automated anomaly detection in a single tool. Existing tools either focus on lab profiling or lack intelligent alerting features [15], [16]. The proposed solution bridges this gap by introducing a low-overhead, continuous monitoring system that adapts to Android's fragmented ecosystem and supports developers in maintaining high performance standards in production environments.

1.5 Structure

The thesis is organized as follows.

Chapter 2 provides a review of previous research and related work on mobile application performance monitoring and anomaly detection. It discusses existing

profiling and monitoring approaches, summarizes key findings, and highlights the research gap that motivates this study.

Chapter 3 provides a detailed description of the design and development of the proposed SDK. It explains the system architecture, the process of collecting performance metrics on Android devices, the transmission of data to the backend server, and the implementation of the anomaly detection mechanism.

Chapter 4 presents the evaluation of the proposed solution. It outlines the experimental setup, describes the conducted tests, and analyzes the obtained results to assess the SDK's performance and anomaly detection accuracy.

Finally, Chapter 5 concludes the thesis by summarizing the main findings, discussing their significance, and showing potential directions for future work.

Chapter 2

Literature Review

This chapter provides an overview of existing research related to mobile applications profiling and monitoring. It examines the methods used to collect performance metrics and the various types of metrics that are considered important for mobile application development. In addition, it discusses different approaches to anomaly detection, providing a foundation for selecting the most suitable method for this study.

The review is divided into three main parts:

- Section 2.1 compares two main approaches to collect performance metrics in mobile applications: pre-release profiling and post-release monitoring. This section outlines their main ideas, methodologies, and trade-offs, showing the advantages and limitations of each approach.
- Section 2.2 focuses on the metrics used in previous research, summarizing which ones are most frequently applied and explaining how they contribute to the assessment of application performance and user experience.
- Section 2.3 provides an overview of existing anomaly detection approaches,

ranging from simple threshold-based methods to advanced statistical and machine learning algorithms.

Together, these sections show the current state of research, providing a foundation for identifying the gaps that the the proposed SDK aims to address.

2.1 Performance monitoring: Concepts and Context

Performance is a critical aspect in mobile application development, as it directly affects user experience and retention. Performance issues such as slow response times, high energy consumption, or UI lags can lead to poor user experience, which may often result in negative ratings and uninstalls. The ability to automatically detect such performance problems can significantly improve application quality and positively impact user satisfaction, as shown by Tang *et al.* [1]. Profiling and monitoring are the two main strategies used to understand and improve application performance. Profiling can be used during development or testing and provides deep insights through high-resolution data collection. Monitoring, in contrast, takes place after the application is released and runs on real user devices. It must introduce as little overhead as possible to prevent any interference with normal app usage.

The Android ecosystem adds complexity to performance evaluation due to fragmentation - the wide variety of hardware, operating system versions, and user behaviors. [4]. This diversity makes it difficult to reproduce rare or device-specific issues found in the wild. The same app may behave differently on various devices depending on their environment, OS version, and hardware specifications

[3]. This problem motivates research into tools and methods for collecting and analyzing mobile performance data effectively.

2.2 Techniques for Performance Data Collection

2.2.1 Pre-release Profiling Techniques

Several tools have been developed to support detailed performance profiling during development. One example is ProfileDroid by Wei *et al.* [5], which uses a multi-layer approach to profiling. Their method involves gathering data from four layers: application metadata, user interface interaction traces, operating system metrics (such as CPU or memory usage), and network activity. This allows developers to observe how different components interact. For example, how a UI event may trigger a CPU spike or a network request. The main strength of ProfileDroid is its ability to correlate performance across layers, giving a complete system-level view. However, its high overhead and dependence on a controlled environment limit its practical use on real devices.

Another relevant project is AndroStep [6], which focuses on analyzing Android storage performance during I/O operations. The authors found that existing Android benchmarking tools did not simulate realistic app behavior, especially missing operations like `fsync()` followed by 4 KB random writes — a common and essential operation used by SQLite [6]. Their tool provides valuable insights for benchmarking Android storage, but it only covers storage metrics and requires manual execution, making it useful only for testing environment.

PowDroid, introduced by Bouaffar *et al.* in 2021 [7], was designed to measure the energy consumption of applications. However, like most pre-release

tools, it works only under controlled conditions, which makes it difficult to test applications on a large range of real devices.

In the field of UI performance, a new state-of-the-art work, GuiWatcher, was proposed by Liu *et al.* [17]. They presented an advanced way to detect UI lags and freezes. GUIWatcher uses computer vision techniques on screen recordings and is able to detect three types of problematic frames: "junky" frames (dropped or delayed ones), long loading frames (lengthy transition), and frozen frames. This method reflects what users actually see rather than relying only on timing logs. However, it is computationally expensive and suitable only for use during development or continuous integration testing. It also raises privacy concerns since it requires recording the device screen.

Finally, ADEL by Zhang *et al.* [18] provides a method to track network performance and detect energy leaks in Android applications by extending the platform with taint-tracking to trace how downloaded data is used within apps. It identifies unnecessary network activity that increases energy consumption. However, the approach is not suitable for continuous monitoring, as it requires modifications to the Android system and introduces high overhead, making it impractical for deployment on real user devices.

2.2.2 Post-release Monitoring Techniques

The main idea of post-release profiling is to collect metrics from apps as they run on real user devices. The main advantage of this approach is that it captures realistic usage patterns and real-world diversity that are hard to reproduce in a lab. The main challenge is to minimize overhead to avoid affecting user experience.

One of the first studies in this area was AppInsight [8], which introduced a

method for instrumenting applications without modifying their source code. The framework automatically injects instrumentation into the compiled bytecode and collects runtime traces. By collecting traces from 30 users over several months, the authors were able to identify performance bottlenecks that were not visible during development. However, this approach is limited because developers cannot easily add custom metrics.

Another example is Hubble [9], which is currently integrated into all Huawei devices. The main feature of Hubble is that it monitors the performance in an efficient in-memory buffer, where older data is constantly overwritten [9]. The relevant metrics are only persisted when a performance issue is detected, providing engineers with detailed runtime traces right before an anomaly occurs. This approach combines high efficiency with low storage cost.

PerfProbe [11] is another post-release performance profiling tool that provides cross-layer insights to identify the root causes of performance issues. It monitors both application and system layers on mobile devices, collecting runtime traces with low overhead. Using statistical analysis, PerfProbe detects critical functions that deviate from normal performance and correlates them with system-level resource factors such as CPU, memory, or I/O usage. This allows developers to understand not only where the slowdown occurs but also why. In experiments with 22 Android apps, PerfProbe successfully diagnosed real-world issues and helped developers to decrease the user-perceived latency of 6 applications by 32-86% [11]

Another important work is Panappticon [10], a lightweight tracing system that captures performance across application, system, and kernel layers. It automatically identifies critical execution paths by linking user inputs to display updates. Unlike AppInsight, Panappticon can detect problems caused by interac-

tions between multiple apps or system components. A one-month study showed it had only about 6% performance overhead, proving it suitable for continuous real-world monitoring.

2.3 Performance Metrics: What to Measure and Why

Performance metrics are the foundation of profiling and anomaly detection. Commonly used metrics include:

- **CPU usage:** High CPU utilization often leads to reduced responsiveness. Overloaded threads or inefficient scheduling can cause UI lag. Studies have shown CPU contention to be a frequent cause of poor user experience on Android [19].
- **Memory usage:** Memory consumption is critical for app stability and avoiding resource leaks. An app that uses excessive RAM can trigger the system's low-memory killer or frequent garbage collection pauses. Developers, therefore, profile heap usage to catch allocation bottlenecks and memory leaks [20]. Keeping memory usage in check helps prevent crashes and ensures smooth operation as the app scales in features or data.
- **I/O operations:** Storage I/O significantly affects latency. This is especially true on mobile devices with slower flash storage or SD cards. For example, opening a new screen can take twice as long as normal due to slow disk I/O [6].
- **Energy consumption:** Battery life is one of the most important factors

for users, because they are likely to notice and uninstall apps that drain the battery excessively. In fact, surveys show battery life often tops performance concerns for smartphone users, and many will avoid apps they perceive as energy-greedy [21]. Therefore, profiling energy usage and detecting “energy leaks” is a key part of mobile optimization.

- **UI responsiveness:** This metric reflects how quickly the user interface responds to user actions. Even when CPU, memory, and other performance indicators appear normal, delayed or uneven UI updates can significantly reduce user satisfaction. Both research and industry standards emphasize the importance of minimizing user-perceived latency. For instance, according to Google’s RAIL model, if a response takes longer than approximately one second, the user’s attention is likely to shift elsewhere [11]. Therefore, smooth scrolling, quick startup, and minimal frame drops are essential for maintaining a positive user experience.

Many tools combine several metrics to provide a comprehensive view. For example, ProfileDroid and Panappticon correlate UI and system activity, while PerfProbe connect CPU and I/O data with execution paths. The choice of metrics depends on the study goal: detailed pre-release profiling or lightweight post-release monitoring.

2.4 Anomaly detection

2.5 Anomaly Detection in Performance Data

Anomalies are deviations from normal behavior that may indicate performance problems, inefficiencies, or bugs. In mobile systems, these can appear as UI freezes, CPU spikes, or sudden battery drain. Although there are few studies focusing specifically on Android performance anomalies, there is extensive research on anomaly detection in system metrics and traces in general.

Both Chandola *et al.* and Schimdl *et al.* [12], [13] focus on structuring and comparing various anomaly detection methods and how they can be applied most efficiently. Chandola describes classic techniques, such as classification-based, clustering-based, nearest-neighbor-based, statistical, etc., providing one of the earliest comprehensive overviews in the field. Schimdl *et al.*, being a more recent study, extend this work by evaluating 71 algorithms and classifying 158 methods into seven major groups: classic ML, signal analysis, stochastic learning, statistics, outlier detection, data mining, and deep learning. Their study focuses on state-of-the-art approaches and highlights how modern methods compare in terms of accuracy and applicability. Both works emphasize that the effectiveness of anomaly detection methods depends on data characteristics and task context, and together they provide valuable insights for selecting relevant methods for anomaly detection in time series of performance metrics.

Another notable example is the work by Ibidunmoye *et al.* [14], where they discussed various methods for detecting and analyzing anomalies in performance metric streams from cloud and web systems. In their work, they compared traditional threshold-based and model-driven approaches with newer adaptive and

data-driven techniques. The authors highlighted the limitations of static models, such as high false-alarm rates and poor adaptability to changing workloads. They emphasized the need for online, unsupervised methods that can adjust to dynamic environments. Their study provides a broad overview of existing anomaly detection strategies and shows how adaptive learning can improve the accuracy and reliability of performance monitoring systems.

2.5.1 Mobile-Specific Gaps and Considerations

While anomaly detection is well explored in cloud and enterprise systems, mobile environments add unique constraints such as hardware diversity, limited resources, and privacy concerns. Systems like Carat [22] demonstrate the unique approach used in mobile anomaly detection. Carat’s main advantage is its scalability and flexibility. By collecting data from many users, it builds a baseline of normal energy usage for each application and identifies outliers — such as a specific app instance on one device or an entire app version that behaves differently from others [22]. This crowd-based method manages the diversity of devices through statistical normalization, for example, by comparing an app’s performance with that of similar devices.

Earlier, Schmidt *et al.* [23] proposed combining lightweight on-device feature extraction with remote anomaly analysis. Although their study was based on older Symbian devices, the architecture remains relevant: it reduces device overhead by moving complex analysis to the cloud, which is still a practical design for mobile anomaly detection today.

2.6 Summary

This chapter reviewed existing research on mobile performance profiling, monitoring, and anomaly detection. It showed that current mobile performance analysis tools can be divided into two main categories: pre-release profiling and post-release monitoring. Firstly, several pre-release tools were discussed, such as ProfileDroid, AndroStep and PowDroid, showing both their strengths and weaknesses. The main advantage of these tools is that they are deeply integrated into the system and can collect a wide range of metrics to understand causes of performance problems later. They are especially useful when used on developers' rooted phones, which allow full system access for debugging. However, they also come with several limitations: they produce large amounts of traces and logs, which causes high overhead and makes them practically unsuitable for use on real users' devices. Moreover, they require system modification or the connection of an external debuggin tool. Post-release monitoring systems, including AppInsight, PerfProbe and Hubble solve this problem by collecting performance data directly from real users with minimal overhead. This gives a more realistic picture of how applications behave under everyday conditions.

The chapter also discussed the classification of performance metrics used in profiling and monitoring of Android applications. It covered common metric categories such as CPU usage, memory consumption, I/O operations, energy usage, and UI responsiveness. Each of these metrics plays a specific role in understanding application performance. For example, CPU and memory metrics help identify resource bottlenecks, while UI and energy metrics are directly related to user experience and device usability. Reviewing these classifications helps determine which metrics are most relevant for detecting performance issues and

supports the design of an effective monitoring system.

The final part of the chapter discussed existing types of anomaly detection methods and how these techniques can be applied to time series of performance metrics. It provided a summary of current detection approaches and outlined how to select the most appropriate methods for analyzing performance data.

In conclusion, despite significant progress in both profiling and anomaly detection, there is still no unified solution that combines continuous, low-overhead performance monitoring with automated cloud-based anomaly detection specifically designed for mobile applications. This gap motivates the development of the proposed SDK, which aims to integrate these two areas into a single, efficient system for Android applications.

Chapter 3

Methodology

3.1 Introduction

This chapter describes the design and development of the proposed Software Development Kit (SDK) for Android applications. Based on the research gap identified in Chapter 2, there is no existing solution that combines continuous post-release monitoring with automated cloud-based anomaly detection. Therefore, the aim of this methodology is to build a system that fills this gap.

The chapter explains the general structure of the research and the technical implementation. It covers the system requirements, the architecture of the solution, the specific metrics selected for collection, and the approach used for anomaly detection.

3.2 System requirements and constraints

To successfully monitor applications on real user devices, the system must follow specific design rules.

- **Low Overhead:** The most important requirement is that the SDK must not slow down the host application. As noted in the literature, tools with high overhead affect the user experience and are not suitable for real-world use. The data collection process must be lightweight.
- **Energy Efficiency:** Battery life is a top concern for users. Therefore, the SDK should not send data continuously over the network. Instead, it must buffer data locally and send it in batches to save energy.
- **Robustness against Fragmentation:** Android devices have very different hardware and OS versions. The SDK must work consistently across different devices without causing crashes.
- **Privacy:** The system focuses only on performance metrics (like CPU and RAM). It assumes that collecting user-identifying information is not necessary for performance profiling, which simplifies privacy compliance

3.3 High-Level System Architecture

The system follows a client-server architecture. It consists of two main components: the Mobile SDK (Client) and the Backend Server.

1. **The Mobile SDK:** This is a library integrated into the Android application. It runs in the background, collects performance data, and sends it to the backend server.
2. **The Backend Server:** Receives the data, stores it in a database, runs the anomaly detection algorithms and notifies developers of any detected issues.

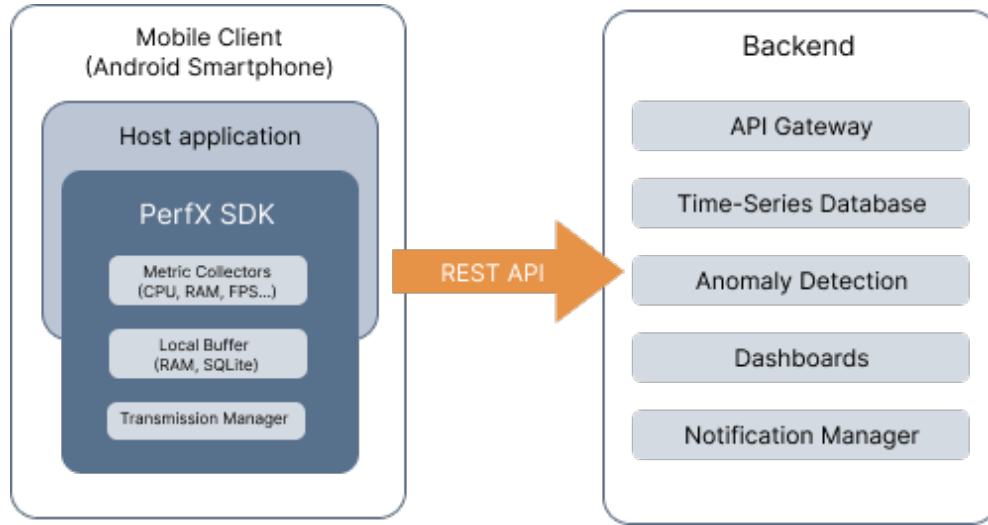


Fig. 3.1. High-Level System Architecture

3.4 Android SDK Design

3.4.1 Metrics Selection and Collection

Based on the literature review, a set of performance metrics was selected because they have a direct impact on user experience and application stability. The goal of the proposed SDK is to collect these metrics in a structured and consistent way, while introducing minimal runtime overhead.

Metric representation model

All collected measurements are represented using a unified metric model. Each metric is defined by:

- a unique identifier, which specifies the metric type;
- a timestamp, which indicates when the measurement was taken;
- screen name, which indicates the currently visible activity in the application;

- a numeric value, which represents the measured quantity.

This representation ensures that different types of metrics can be processed uniformly by the backend system. Using a common structure also simplifies data storage and analysis.

Timestamping and sampling strategy

All metrics are collected using a monotonic timestamp. This avoids issues related to system clock changes and ensures that time differences between measurements are reliable.

Metrics are collected using two strategies:

- **Periodic sampling** for continuous metrics such as CPU usage, memory usage, and frame time.
- **Event-based collection** for discrete events such as crashes, ANR occurrences, application startup and custom user-defined events.

CPU usage

CPU usage reflects how much processing time the application consumes relative to the system. It is computed using the difference in CPU time used by the application process compared to the total CPU time over a sampling interval.

Let ΔP be the increase in process CPU time and ΔS be the increase in total system CPU time:

$$CPU\% = \frac{\Delta P}{\Delta S} \cdot 100 \quad (3.1)$$

This relative measurement makes CPU usage comparable across devices with different hardware configurations.

Memory usage

Memory usage is measured using the proportional memory footprint of the application. The selected value represents how much physical memory is effectively used by the application, including shared memory.

$$MemoryUsage(MB) = \frac{PSS(KB)}{1024} \quad (3.2)$$

This metric is important for detecting memory leaks and excessive memory consumption that can lead to application slowdowns or termination by the operating system.

Frame time and UI performance

UI performance is evaluated by measuring frame time, which represents the duration required to render a single frame.

If t_i and t_{i-1} are timestamps of consecutive frames:

$$FrameTime(ms) = \frac{t_i - t_{i-1}}{10^6} \quad (3.3)$$

High frame times indicate UI lag. For example, on a 60 Hz display, frame times exceeding approximately 16.67 ms may cause visible stuttering. This metric directly reflects user-perceived responsiveness.

$$FPS = \frac{1000}{FrameTime(ms)} \quad (3.4)$$

Startup time

Startup time measures how long it takes for the application to become visually responsive after launch. It is defined as the time difference between application start and the first fully rendered frame.

$$StartupTime(ms) = t_{\text{firstFrame}} - t_{\text{start}} \quad (3.5)$$

Long startup times negatively affect first impressions and user retention.

Stability-related metrics

Stability is monitored using event-based metrics:

- **Crash events**, which represent unhandled runtime exceptions.
- **ANR (Application Not Responding) events**, which indicate prolonged blocking of the main thread.

Metric aggregation and transmission

Collected metrics are stored locally in a buffer and transmitted in batches. Batching reduces network overhead since opening a network connection is energy-intensive, so reducing the frequency of transmissions helps conserve battery life.

A batch is sent when either a predefined number of metrics is reached or a time threshold expires:

$$SendBatch = \begin{cases} true & \text{if } size \geq N \vee (t - t_{\text{last}}) \geq W \\ false & \text{otherwise} \end{cases} \quad (3.6)$$

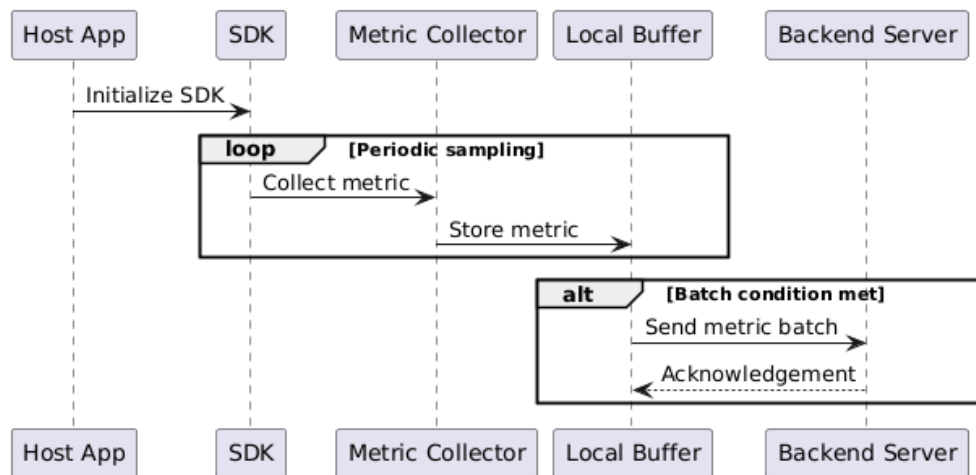


Fig. 3.2. Sequence Diagram of SDK Data Collection and Transmission

3.5 Backend Server Design

The backend server is responsible for receiving performance metrics from Android devices, storing them, and performing anomaly detection. The server-side design focuses on scalability, reliability, and the ability to process large volumes of time-dependent data.

3.5.1 Data Storage

The system uses a time-series database for storing performance metrics. This type of database is chosen because performance metrics naturally form time-series data, where each value is associated with a timestamp. Time-series databases are optimized for frequent writes, range queries over time, and aggregation operations, which are common in performance monitoring systems.

Choice of database

A dedicated time-series database is used instead of a traditional relational database. This choice is motivated by the following reasons:

- Efficient storage of large volumes of timestamped data.
- Fast aggregation queries (e.g., averages, percentiles, trends).
- Built-in support for retention policies.
- Good scalability for increasing numbers of devices and metrics.

Database schema

Each stored metric contains the following logical fields:

- Metric identifier (e.g., CPU usage, memory usage, frame time).
- Timestamp of measurement.
- Numeric value of the metric.
- Application context information (such as activity or screen name).
- Device and application metadata (for example, OS version or app version).

Data retention policies

To limit storage growth, retention policies are applied. Raw metric data is stored for a limited time window, after which it is deleted.

Retention policies help balance storage costs and long-term analysis needs.

3.5.2 Anomaly Detection Algorithms

Anomaly detection is used to identify abnormal patterns in performance metrics that may indicate performance degradation, instability, or poor user experience. Since the system operates on production data and no labeled anomaly data is available, the selected methods must work without supervised training. In addition, the detection must be computationally efficient, interpretable, and suitable for continuous backend processing.

Threshold-based methods

Threshold-based anomaly detection relies on predefined limits for each metric. A value is considered anomalous if it exceeds a fixed upper or lower threshold.

$$x_t > T_{\max} \quad \text{or} \quad x_t < T_{\min} \quad (3.7)$$

Advantages:

- Very simple to implement and understand.
- Low computational cost.
- Immediate detection without historical data.

Disadvantages:

- Thresholds must be manually defined.
- Not adaptive to changes in application behavior.
- High risk of false positives or false negatives.

Due to their inflexibility, static thresholds are not sufficient as the main detection method in this work.

Statistical deviation-based methods

Statistical methods detect anomalies by comparing new values against a baseline computed from historical data. Common approaches use measures such as mean, standard deviation, median, or dispersion.

A value is considered anomalous if it deviates significantly from the baseline:

$$|x_t - \mu| > k \cdot \sigma \quad (3.8)$$

Advantages:

- No need for labeled data.
- Adaptive to normal behavior patterns.
- Efficient and suitable for real-time analysis.

Disadvantages:

- Sensitive to outliers if simple averages are used.
- Requires careful selection of window size and parameters.

These methods are well suited for performance metrics such as CPU usage and memory consumption.

Percentile-based methods

Percentile-based detection focuses on extreme values rather than average behavior. A metric is considered anomalous if it exceeds a high percentile of historical values, such as the 95th or 99th percentile.

$$x_t > P_{95} \quad (3.9)$$

Advantages:

- Robust to non-normal and skewed distributions.
- Well suited for tail-latency metrics.
- Easy to interpret.

Disadvantages:

- Requires sufficient historical data.
- May miss gradual performance degradation.

Percentile-based methods are particularly effective for frame time, startup time, and user interaction delays.

Rate-of-change detection

Rate-of-change detection identifies anomalies by analyzing differences between consecutive values rather than absolute values.

$$\Delta x_t = x_t - x_{t-1} \quad (3.10)$$

An anomaly is detected if the change exceeds a predefined threshold.

Advantages:

- Effective for detecting sudden spikes.
- Independent of absolute metric scale.

Disadvantages:

- Does not detect consistently high values.
- Sensitive to noisy data.

This method complements other approaches by detecting abrupt changes in system behavior.

Forecasting-based methods

Forecasting-based methods predict future values of a metric using historical data and flag anomalies when observations fall outside prediction intervals.

Advantages:

- Can model trends and seasonality.
- Suitable for long-term, stable datasets.

Disadvantages:

- Requires model fitting and parameter tuning.
- Higher computational cost.
- Poor performance on noisy, short-lived metrics.

Due to the dynamic and bursty nature of mobile performance metrics, forecasting-based methods are not selected for this work.

Machine learning–based methods

Machine learning approaches use clustering or classification to identify abnormal patterns.

Advantages:

- Capable of detecting complex patterns.

Disadvantages:

- Often require labeled data or extensive training.
- Difficult to interpret.
- Increased system complexity.

These properties make machine learning methods unsuitable for the goals of this thesis.

Selected approach

Considering the system requirements, this work adopts a combination of training-free statistical methods. Rolling-window statistical deviation, percentile-based detection, and rate-of-change analysis are combined to identify anomalies reliably. In addition, correlations between related metrics are considered to reduce false positives.

This hybrid statistical approach provides a practical balance between detection accuracy and system simplicity, making it suitable for continuous performance monitoring in real-world Android applications.

3.6 Testing and Validation

Testing and validation focus on verifying the correctness, performance, and reliability of the proposed system. The SDK is tested by integrating it into sample applications and observing its behavior under different usage scenarios.

Validation includes checking that collected metrics are accurate, that the backend correctly stores and processes incoming data, and that anomaly detection reacts appropriately to abnormal behavior.

3.7 Summary

This chapter described the methodology for developing a performance monitoring SDK. The design prioritizes low overhead and robustness to solve the problems identified in the literature. By collecting metrics on the client side and performing statistical anomaly detection on the server side, the system aims to provide developers with automated insights into application performance. The next chapter will detail the specific code implementation and technical challenges encountered.

Chapter 4

Implementation

...

Chapter 5

Evaluation and Discussion

...

Chapter 6

Conclusion

...

Bibliography cited

- [1] Y. Tang, H. Wang, X. Zhan, *et al.*, “A systematical study on application performance management libraries for apps,” *IEEE Trans. Softw. Eng.*, vol. 48, no. 8, pp. 3044–3065, Aug. 2022, ISSN: 0098-5589. DOI: [10.1109/TSE.2021.3077654](https://doi.org/10.1109/TSE.2021.3077654).
- [2] Firebase. “Firebase performance monitoring.” Accessed: 30 Oct 2025. (2025), [Online]. Available: <https://firebase.google.com/docs/perf-mon>.
- [3] Z. Dong, Y. Zhao, T. Liu, *et al.*, “Same app, different behaviors: Uncovering device-specific behaviors in android apps,” in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE ’24, Sacramento, CA, USA: Association for Computing Machinery, 2024, pp. 2099–2109, ISBN: 9798400712487. DOI: [10.1145/3691620.3695272](https://doi.org/10.1145/3691620.3695272).
- [4] M. Kamran, J. Rashid, and M. Nisar, “Android fragmentation classification, causes, problems and solutions,” *International Journal of Computer Network and Information Security*, vol. 14, pp. 992–999, Sep. 2016.
- [5] X. Wei, L. Gomez, I. Neamtii, and M. Faloutsos, “Profiledroid: Multi-layer profiling of android applications,” Aug. 2012. DOI: [10.1145/2348543.2348563](https://doi.org/10.1145/2348543.2348563).

- [6] S. Jeong, K. Lee, J.-Y. Hwang, S. Lee, and Y. Won, “Androstep: Android storage performance analysis tool,” in *Software Engineering*, 2013.
- [7] F. Bouaffar, O. L. Goaer, and A. Nouredine, “Powdroid: Energy profiling of android applications,” in *2021 36th IEEE/ACM International Conference on Automated Software Engineering Workshops (ASEW)*, 2021, pp. 251–254. DOI: [10.1109/ASEW52652.2021.00055](https://doi.org/10.1109/ASEW52652.2021.00055).
- [8] L. R. Sivalingam, J. Padhye, S. Agarwal, R. Mahajan, I. Obermiller, and S. Shayandeh, “Appinsight: Mobile app performance monitoring in the wild,” Nov. 2012.
- [9] Y. Luo, K. Rodrigues, C. Li, *et al.*, “Hubble: Performance debugging with In-Production, Just-In-Time method tracing on android,” in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, Carlsbad, CA: USENIX Association, Jul. 2022, pp. 787–803, ISBN: 978-1-939133-28-1.
- [10] L. Zhang, D. Bild, R. Dick, Z. Mao, and P. Dinda, “Panappticon: Event-based tracing to measure mobile application and platform performance,” Sep. 2013, pp. 1–10. DOI: [10.1109/CODES-ISSS.2013.6659020](https://doi.org/10.1109/CODES-ISSS.2013.6659020).
- [11] D. K. Hong, A. Nikraves, Z. M. Mao, M. Ketkar, and M. Kishinevsky, “Perfprobe: A systematic, cross-layer performance diagnosis framework for mobile platforms,” in *2019 IEEE/ACM 6th International Conference on Mobile Software Engineering and Systems (MOBILESoft)*, 2019, pp. 50–61. DOI: [10.1109/MOBILESoft.2019.00018](https://doi.org/10.1109/MOBILESoft.2019.00018).

- [12] V. Chandola, A. Banerjee, and V. Kumar, “Anomaly detection: A survey,” *ACM Comput. Surv.*, vol. 41, no. 3, Jul. 2009, ISSN: 0360-0300. DOI: [10.1145/1541880.1541882](https://doi.org/10.1145/1541880.1541882).
- [13] S. Schmidl, P. Wenig, and T. Papenbrock, “Anomaly detection in time series: A comprehensive evaluation,” *Proc. VLDB Endow.*, vol. 15, no. 9, pp. 1779–1797, May 2022, ISSN: 2150-8097. DOI: [10.14778/3538598.3538602](https://doi.org/10.14778/3538598.3538602).
- [14] O. Ibidunmoye, F. Hernandez-Rodriguez, and E. Elmroth, “Performance anomaly detection and bottleneck identification,” *ACM Computing Surveys*, vol. 48, Jun. 2015. DOI: [10.1145/2791120](https://doi.org/10.1145/2791120).
- [15] Google Android. “Android vitals: App quality.” Accessed: 30 Oct 2025. (2025), [Online]. Available: <https://developer.android.com/topic/performance/vitals>.
- [16] Android Developers. “Monitoring app performance.” Accessed: 30 Oct 2025. (2022), [Online]. Available: <https://medium.com/androiddevelopers/monitoring-app-performance-ebf9af4ebe36>.
- [17] W. Liu, F. Lin, L. Guo, T.-H. Chen, and A. Hassan, “Guiwatcher: Automatically detecting gui lags by analyzing mobile application screencasts,” Feb. 2025. DOI: [10.48550/arXiv.2502.04202](https://doi.org/10.48550/arXiv.2502.04202).
- [18] L. Zhang, M. Gordon, R. Dick, Z. Mao, P. Dinda, and L. Yang, “Adel: An automatic detector of energy leaks for smartphone applications,” Oct. 2012, pp. 363–372. DOI: [10.1145/2380445.2380503](https://doi.org/10.1145/2380445.2380503).

- [19] J. Fu, Y. Wang, Y. Zhou, and X. Wang, “How resource utilization influences ui responsiveness of android software,” *Information and Software Technology*, vol. 141, p. 106 728, Sep. 2021. DOI: [10.1016/j.infsof.2021.106728](https://doi.org/10.1016/j.infsof.2021.106728).
- [20] B. Li, Q. Zhao, S. Jiao, and X. Liu, “Droidperf: Profiling memory objects on android devices,” Jul. 2023, pp. 1–15. DOI: [10.1145/3570361.3592503](https://doi.org/10.1145/3570361.3592503).
- [21] R. Rua and J. Saraiva, “A large-scale empirical study on mobile performance: Energy, run-time and memory,” *Empirical Softw. Engg.*, vol. 29, no. 1, Dec. 2023, ISSN: 1382-3256. DOI: [10.1007/s10664-023-10391-y](https://doi.org/10.1007/s10664-023-10391-y).
- [22] A. Oliner, A. Iyer, I. Stoica, E. Lagerspetz, and S. Tarkoma, “Carat: Collaborative energy diagnosis for mobile devices,” Nov. 2013. DOI: [10.1145/2517351.2517354](https://doi.org/10.1145/2517351.2517354).
- [23] A.-D. Schmidt, F. Peters, F. Lamour, S. Albayrak, and S. Camtepe, “Monitoring smartphones for anomaly detection,” vol. 14, Feb. 2008, p. 40. DOI: [10.1007/s11036-008-0113-x](https://doi.org/10.1007/s11036-008-0113-x).